

## Aula 14 - Execução dos exemplos de testes

### 1. Testes de Integração com Banco de Dados

Para executar o teste de integração com banco de dados usando o PHPUnit, nós precisamos seguir um passo a passo que vai desde a organização dos arquivos até o comando de execução no terminal. Abaixo um passo a passo de como fazer isso:

#### **Passo 1:** Organizar a Estrutura de Pastas e Arquivos

Para o PHPUnit funcionar perfeitamente, garanta que os arquivos criados nos passos anteriores estejam nesta estrutura de pastas:

```
C:\meu-projeto\  
├── src\  
│   └── UserRepository.php  
├── tests\  
│   └── UserRepositoryTest.php  
├── vendor\  
├── composer.json  
└── phpunit.xml
```

#### **Passo 2:** Configurando o PHP

O Windows não vem com o driver do SQLite ativo por padrão. Se você tentar rodar o teste agora, ele vai falhar. Vamos resolver isso:

1. Descubra onde o seu PHP está instalado (ex: C:\xampp\php\ ou C:\php\).
2. Abra essa pasta e procure pelo arquivo chamado php.ini. Abra-o com o Bloco de Notas ou VS Code.
3. Aperte Ctrl + F (Localizar) e digite: pdo\_sqlite
4. Você encontrará uma linha parecida com isto: ;extension=pdo\_sqlite
5. Remova o ponto e vírgula (;) do início da linha para ativá-la. Ela deve ficar exatamente assim:  
*extension=pdo\_sqlite*
6. (Opcional) Aproveite e faça o mesmo com a linha ;extension=sqlite3 (removendo o ;).
7. Salve e feche o arquivo.

#### **Passo 3:** Instalar as Dependências pelo Prompt de Comando

Abra o Prompt de Comando (cmd) ou o PowerShell no Windows, navegue até a pasta do seu projeto e instale o PHPUnit caso não esteja instalado:

```
cd C:\meu-projeto  
composer require --dev phpunit/phpunit
```

#### **Passo 4:** Configurar o arquivo phpunit.xml

Na raiz da pasta C:\meu-projeto, certifique-se de ter o arquivo phpunit.xml com o seguinte conteúdo para o Windows saber onde procurar os testes:

```
XML  
<?xml version="1.0" encoding="UTF-8"?>  
<phpunit bootstrap="vendor/autoload.php" colors="true">  
  <testsuites>  
    <testsuite name="Integration">  
      <directory>tests</directory>  
    </testsuite>  
  </testsuites>  
</phpunit>
```

#### **Passo 5:** Configurar o composer.json

Abra o arquivo composer.json (que fica na raiz do seu projeto C:\meu-projeto\) e adicione a seção autoload para dizer ao PHP que tudo o que estiver na pasta src/ deve ser carregado automaticamente.

Deixe o seu arquivo configurado assim:

JSON

```
{
  "require": {
    "guzzlehttp/guzzle": "^7.8"
  },
  "require-dev": {
    "phpunit/phpunit": "^10.0"
  },
  "autoload": {
    "psr-4": {
      "": "src/"
    }
  }
}
```

#### **Passo 6:** Atualizar o Autoload do Composer

Toda vez que alteramos a seção autoload no arquivo composer.json, precisamos avisar o Composer para reconstruir o mapa de arquivos.

Abra o Prompt de Comando ou PowerShell, navegue até a pasta do projeto e digite:

```
cd C:\meu-projeto
```

```
composer dump-autoload
```

#### **Passo 7:** Executar o Teste

Ainda no Prompt de Comando/PowerShell dentro da pasta do projeto, execute o binário do PHPUnit usando barras invertidas (padrão do Windows):

DOS

```
.\vendor\bin\phpunit tests\UserRepositoryTest.php
```

#### **Passo 8:** Analisar o Resultado

Agora que o driver do SQLite está ativo, o PHP conseguirá criar o banco em memória e você verá a tela de sucesso:

PHPUnit 10.x.x by Sebastian Bergmann and contributors.

Runtime: PHP 8.2.x (com pdo\_sqlite ativo)

Configuration: C:\meu-projeto\phpunit.xml

. 1 / 1 (100%)

Time: 00:00.018, Memory: 6.00 MB

OK (1 test, 2 assertions)

O ponto . indica que o teste passou e a mensagem OK confirma que a gravação e a busca no banco SQLite temporário funcionaram perfeitamente!

**Dica de ouro:** Como estamos usando o `sqlite::memory:` dentro do método `setUp()`, você não precisa se preocupar em criar ou configurar um banco de dados MySQL na sua máquina antes de rodar o comando. O PHPUnit cria o banco na memória RAM, roda o teste e apaga tudo sozinho em milissegundos!

---

## 2. Testes de Integração com APIs Externas (Ex: ViaCEP)

Para executar o teste de integração com uma API externa usando o PHPUnit e o Guzzle, o processo é muito parecido com o do banco de dados, mas com uma grande vantagem: como estamos simulando a internet localmente (usando Mocks), você não corre o risco de o teste falhar se a API do ViaCEP cair ou se você ficar sem Wi-Fi. Abaixo o passo a passo detalhado:

### **Passo 1:** Organizar a Estrutura de Pastas e Arquivos

Para que o teste funcione, precisamos garantir que o arquivo de teste e as classes do seu sistema estejam nos lugares certos. A estrutura ideal fica assim:

```
meu-projeto/
├── src/
│   ├── ViaCepService.php    (A classe que faz a requisição para o ViaCEP)
│   └── Endereco.php        (O objeto que representa o endereço retornado)
├── tests/
│   └── ViaCepServiceTest.php
├── vendor/
├── composer.json
└── phpunit.xml
```

### **Passo 2:** Instalar o Guzzle HTTP (Caso não tenha)

Como o teste utiliza o cliente HTTP Guzzle e seus componentes de simulação (MockHandler), você precisa garantir que ele está instalado no projeto. Se ainda não estiver, rode no terminal:

```
composer require guzzlehttp/guzzle
```

**Nota:** Certifique-se de que o PHPUnit também está instalado (conforme fizemos no exemplo 1 via `composer require --dev phpunit/phpunit`).

### **Passo 3:** Executar o Comando no Terminal

Abra o terminal na raiz do seu projeto e execute o PHPUnit apontando diretamente para o arquivo de teste da API:

```
./vendor/bin/phpunit tests/ViaCepServiceTest.php
```

### **Passo 4:** Entender o Fluxo de Execução (O que acontece por trás dos panos)

Para entender por que esse teste roda tão rápido (geralmente em menos de 10 milissegundos), veja o que o PHPUnit faz quando você aperta o Enter:

[Início do Teste]

- ▼
  1. Cria uma resposta JSON falsa na memória (Simulando o ViaCEP).
- ▼
  2. Intercepta o cliente Guzzle usando o MockHandler.
- ▼
  3. O ViaCepService tenta "viajar para a internet" buscar o CEP...
- ▼
  4. O Guzzle intercepta a rota e devolve IMEDIATAMENTE o JSON falso.
- ▼
  5. O ViaCepService transforma o JSON no objeto "Endereco".



6. O PHPUnit valida se os métodos getLogradouro() e getCidade() batem com o JSON.



[Fim do Teste: Sucesso OK]

### **Passo 5:** Analisar o Resultado

Se o teste **PASSAR**:

Como não há conexão real com a internet, o teste será instantâneo. Você verá a confirmação de sucesso:

PHPUnit 10.x.x by Sebastian Bergmann and contributors.

. 1 / 1 (100%)

Time: 00:00.008, Memory: 6.00 MB

OK (1 test, 2 assertions)

Se o teste **FALHAR**:

Se o teste falhar, geralmente significa que a sua classe ViaCepService tem algum erro na hora de mapear o JSON. O PHPUnit vai te avisar exatamente onde está a divergência:

There was 1 failure:

1) ViaCepServiceTest::test\_deve\_retornar\_o\_endereco\_correto\_ao\_buscar\_por\_cep  
Failed asserting that 'Bairro Errado' matches expected 'Praça da Sé'.

FAILURES!

Tests: 1, Assertions: 2, Failures: 1.

Se isso acontecer, basta ajustar a lógica dentro da sua classe ViaCepService (onde você lê o JSON) e rodar o comando novamente!